

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №3
по дисциплине «Алгоритмические основы компьютерной графики»
«Визуализация физического процесса»

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Курочкин М. А.

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Описание физического процесса	5
3 Визуализация	8
4 Результаты работы	10
Заключение	12
Список использованной литературы	13

Введение

Компьютерная графика - это область информатики, которая занимается созданием и обработкой графических изображений. Одним из ее разделов является анимация, которая «оживляет» статическую картинку, двигая ее компоненты. Существуют несколько типов анимации:

- анимация захвата движения;
- покадровая анимация;
- анимация с помощью специализированного программного обеспечения;
- процедурная анимация.

Процедурная анимация - это метод, при котором движение объектов автоматически генерируется по установленным правилам, физическим законам и ограничениям.

Цель данной работы - создание визуализации физического процесса горения одной конфорки на газовой плите. В качестве образца будет использован видеоролик, фиксирующий процесс.

Основные характеристики визуализации реального физического процесса включают:

- Временную протяженность: процесс горения изменяется во времени, что требует создания анимации.
- Статические и динамические сущности: в процессе горения есть как неизменные сущности, так и те, которые меняют свое положение, цвет и размеры.

1 Постановка задачи

Дано:

Видеоролик физического процесса - горения газа в газовой конфорке на средней мощности.

Необходимо:

- Проанализировать видеоролик, выделить отдельные этапы в физическом процессе.
- Разработать метод визуализации для каждого этапа.
- Реализовать визуализацию процесса при помощи графической библиотеки.

2 Описание физического процесса

Физический процесс можно разделить на статическую и динамическую части.

Статическая часть: круглая конфорка черного цвета, которая для позиции камеры представляет эллипс; металлическое основание со следами нагара; плита белого цвета, в левом нижнем углу которой черная трещина; черная решетка для посуды.

Динамическая часть:

1. Языки пламени, образующиеся при сгорании газа в газовой горелке. В конфорке есть 26 одинаковых отверстий, через которые выходит газ. В процессе сгорания газа формируются языки пламени. Из них малые языки пламени выходят из двух, из 23 - средние, из одной языка пламени не видно. Языки пламени, которые находятся на заднем плане для наблюдателя, выглядят более вытянутыми вверх из-за разницы в ракурсе. Пример языков пламени представлен на рисунке 1. Основные элементы:

• **Цвет пламени**

- Основание пламени насыщенного голубого цвета.
- Верхняя часть пламени светло-голубого оттенка, полупрозрачная.

• **Форма пламени:**

- Слегка изогнутые вытянутые овалы, которые каплеобразно сужаются к вершине.
- Форма пламени изменяется: вершины языков слегка вытягиваются и сужаются, основание при этом стабильно.

• **Высота пламени:**

- Средняя высота составляет 5-10 пикселей.
- Высота одного языка пламени колеблется в течение процесса с амплитудой 1-2 пикселя из-за нестабильности газа.
- Частота колебаний около 3-5 колебаний в секунду.

• **Угол наклона пламени:**

- Языки пламени наклонены наружу под углом $3 - 5^\circ$ от вертикали. Из-за разницы в ракурсе у языков спереди и сзади наклон почти незаметен.
- Амплитуда изменения угла наклона около $1 - 2^\circ$ от вертикали.



Рис. 1. Кадр из видеоролика, 0.1 с

2. Искры возникают в основании или на вершинах языков пламени. Они появляются в случайные моменты времени из-за попадания в пламя посторонних частиц. Пример искр представлен на рисунках 2- 3.

- **Цвет искр:**

- Насыщенный оранжевый, по краям переходящий в белый.
- Прозрачность цвета меняется в зависимости от этапа жизни искры. Пик яркости наступает в середине жизни искры.

- **Размеры и форма искр:**

- Искры овальной формы, иногда вытянутые по форме верхушки языка пламени.
- Длина искры составляет 3-5 пикселей.

- **Время жизни:**

- Среднее время жизни искры составляет 0.05-0.2 секунды.
- В течение времени своей жизни искры плавно разгораются и затухают.

- **Периодичность появления:**

- За одну секунду появляется около 3-4 искр.
- Около одного языка пламени в конкретный момент времени появляется одна искра.



Рис. 2. Кадр из видеоролика, 0.1 с



Рис. 3. Кадр из видеоролика, 0.4 с

3 Визуализация

Для визуализации данного физического процесса был проведен следующий анализ и моделирование:

Процесс был разделен на две ключевые составляющие:

- Визуализация пламени, исходящего из одного отверстия конфорки.
- Визуализация искр, возникающих в конкретные моменты времени.
- Добавления пламени и искр на сцену с предварительно загруженным изображением статической части, взятой из предоставленного видео.

Первый этап — создание текстур пламени и искр, которые потом добавляются на сцену. На сцене предварительно размещается фоновое изображение конфорки, полученное из исходного видео.

Для визуализации нужно разработать 2 вида текстур: один вид - для передачи языков пламени, второй - для визуализации искр. Первый вид текстур включает 24 элемента, на которых показаны ракурсы каждого языка пламени. Второй вид текстур включает 33 элемента, где каждый элемент - это отдельная вспышка, произошедшая за время видеофиксации процесса.

Все изображения обработаны в графическом редакторе paint.net для создания реалистичных изображений и обеспечения прозрачного фона. Примеры текстур представлены на рисунке 4.

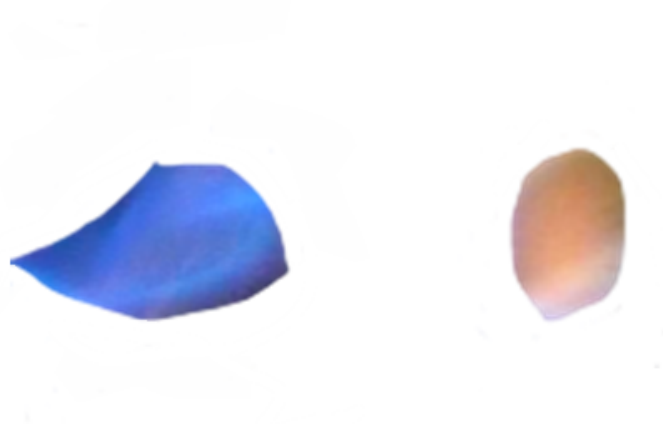


Рис. 4. Примеры текстуры огня и искры

Второй этап — анимация сцены с помощью Python. Программа также использует библиотеку Pygame для отрисовки.

Структура программы

1. Инициализация библиотек, загрузка изображений, необходимых для визуализации.
2. Настройка основных параметров:
 - `DURATION` - длительность программы в секундах (10.66 секунд).
 - `FPS` - частота кадров в секунду (50).
 - `FLAMES_CONFIG`, `SPARKS_CONFIG` - записи о каждом языке пламени и каждой искре с необходимыми характеристиками.
 - `start_time` - время начала анимации.
 - `clock` - объект для управления временем в Pygame.
3. Инициализация языков пламени и искр с помощью конструкторов соответствующих классов:
 - Класс `FlameLayer` реализован для управления языками пламени. Каждый кадр обновляется случайное смещение позиции и масштаба для создания эффекта дрожания. Каждый экземпляр класса представляет отдельный слой пламени с заданной позицией на экране, коэффициентом масштабирования текстуры, непрозрачностью, амплитудой случайного дрожания и изменения масштаба, параметрами приближенного размытия.
 - Класс `SparkParticle` моделирует искру с заданными координатами, размером, временем жизни, пиковой непрозрачностью и параметрами приближенного размытия.
4. Главный цикл программы, который выполняется на каждом кадре:
 - 4.1. Обработка событий, включая закрытие окна в случае, когда время превышает установленный `DURATION`.
 - 4.2. Отрисовка фоновое изображение.
 - 4.3. Обновление параметров дрожания для всех слоёв пламени и его отрисовка.
 - 4.4. Активация искр по заранее заданному расписанию и их отрисовка.
 - 4.5. Обновление возраста, удаление «умерших» искр.

4 Результаты работы

Результаты сравнения созданной визуализации и исходного видео представлены на рисунках 5- 6. Исходное видео находится сверху, визуализация - снизу.



Рис. 5. Сравнение оригинального видео с результатом



Рис. 6. Сравнение оригинального видео с результатом

Заключение

В ходе данной лабораторной работы была осуществлена визуализация физического процесса горения газа в конфорке на основе анализа соответствующего видеоролика. В ходе анализа были выявлены ключевые характеристики динамической и статической частей процесса. На основе полученных данных была разработана графическая модель с использованием библиотеки Python для визуализации горения газа.

Для достижения более реалистичного отображения пламени применялась техника наложения текстур на изображение. Для улучшения точности визуализации можно реализовать полупрозрачную текстуру с плавным шумом для имитации движения горячего воздуха

Список литературы

- [1] Raphael, Holzer Документация к модулю Pygame — URL: <https://www.pygame.org/contribute.html> (дата обращения: 28.03.2026).

Приложение 1. Код программы

Листинг 1. Полный код программы

```
1 import pygame
2 import random
3 import time
4
5 BACKGROUND_IMAGE = "bg.png"
6 SIZE_COEF = 0.6
7
8 TEXTURE_FRONT = "flame_front_2.png"
9 TEXTURE_FRONT_RIGHT = "flame_front_right.png"
10 TEXTURE_3 = "flame3-7.png"
11 TEXTURE_4 = "flame4.png"
12 TEXTURE_5 = "flame5.png"
13 TEXTURE_6 = "flame6.png"
14 TEXTURE_7 = "flame7.png"
15 TEXTURE_8 = "flame8.png"
16 TEXTURE_9 = "flame9.png"
17 TEXTURE_10 = "flame10.png"
18 TEXTURE_11 = "flame11.png"
19 TEXTURE_12 = "flame12.png"
20 TEXTURE_13 = "flame13.png"
21 TEXTURE_14 = "flame14.png"
22 TEXTURE_15 = "flame15.png"
23 TEXTURE_16 = "flame16.png"
24 TEXTURE_17 = "flame17.png"
25 TEXTURE_18 = "flame18.png"
26 TEXTURE_185 = "flame18.5.png"
27 TEXTURE_19 = "flame19.png"
28 TEXTURE_20 = "flame20.png"
29 TEXTURE_21 = "flame21.png"
30 TEXTURE_22 = "flame22.png"
31 TEXTURE_23 = "flame23.png"
32 TEXTURE_24 = "flame24.png"
33
34 FPS = 50
35 DURATION = 10.66
36
37 SPARKS_CONFIG = [
38 (0.00, "spark_1.png", 750, 425, 0.6, 0.2, 255, 0, 0),
39 (0.16, "spark_2.png", 460, 142, 0.6, 0.2, 255, 0, 0),
40 (1.16, "spark_3.png", 940, 195, 0.7, 0.05, 255, 0, 0),
41 (1.2, "spark_4.png", 288, 280, 0.7, 0.05, 255, 0, 0),
42 (1.42, "spark_5.png", 695, 430, 0.7, 0.05, 200, 0, 0),
43 (1.8, "spark_4.png", 288, 280, 0.7, 0.08, 100, 0, 0),
44 (2.2, "spark_6.png", 970, 210, 0.7, 0.05, 255, 0, 0),
45 (2.42, "spark_7.png", 880, 375, 0.7, 0.05, 255, 0, 0),
46 (3.0, "spark_8.png", 480, 445, 0.7, 0.05, 255, 0, 0),
47 (3.28, "spark_8.png", 480, 445, 0.7, 0.05, 255, 0, 0),
48 (3.4, "spark_9.png", 705, 450, 0.8, 0.05, 255, 0, 0),
49 (3.4, "spark_10.png", 400, 168, 0.7, 0.05, 255, 0, 0),
50 (4.1, "spark_11.png", 830, 140, 0.7, 0.05, 255, 0, 0),
51 (4.33, "spark_12.png", 350, 200, 0.7, 0.1, 255, 0, 0),
52 (4.91, "spark_13.png", 770, 420, 0.8, 0.05, 255, 0, 0),
```

```

53 (4.91, "spark_14.png", 835, 130, 0.7, 0.05, 255, 0, 1),
54 (4.91, "spark_15.png", 343, 145, 0.7, 0.05, 255, 0, 2),
55
56 (5.52, "spark_16.png", 795, 365, 0.7, 0.06, 255, 0, 2),
57 (5.64, "spark_18.png", 390, 168, 0.7, 0.15, 255, 0, 2),
58 (5.84, "spark_19.png", 875, 145, 0.7, 0.05, 255, 0, 2),
59 (6.3, "spark_20.png", 313, 325, 0.7, 0.2, 255, 0, 2),
60 (6.42, "spark_21.png", 480, 400, 0.8, 0.05, 255, 0, 2),
61 (6.84, "spark_22.png", 295, 210, 0.7, 0.05, 255, 0, 2),
62 (7.44, "spark_23.png", 810, 415, 0.8, 0.05, 255, 0, 2),
63 (7.77, "spark_24.png", 960, 230, 0.8, 0.1, 255, 0, 2),
64 (7.8, "spark_25.png", 830, 140, 0.8, 0.05, 255, 0, 2),
65 (8.36, "spark_26.png", 460, 120, 0.8, 0.05, 255, 0, 2),
66 (8.44, "spark_27.png", 710, 432, 0.8, 0.05, 255, 0, 2),
67 (9.04, "spark_28.png", 290, 240, 0.8, 0.05, 255, 0, 2),
68 (9.08, "spark_29.png", 645, 437, 0.8, 0.05, 255, 0, 2),
69 (9.45, "spark_30.png", 615, 113, 0.8, 0.1, 255, 0, 2),
70 (9.72, "spark_31.png", 460, 152, 0.8, 0.1, 255, 0, 2),
71 (10.22, "spark_32.png", 285, 305, 0.8, 0.1, 255, 0, 2),
72 (10.56, "spark_33.png", 615, 110, 0.8, 0.2, 255, 0, 2),
73 ]
74 # Время появления искры секунды( от начала), файл, позиция центра искры, масштаб,
75 # длительность жизни, макс непрозрачность, радиус размытия, количество итераций
  размытий
76
77 RECORD_VIDEO = True
78 VIDEO_OUTPUT = "Мартыненко.mp4"
79 VIDEO_FPS = 50
80
81
82 FLAMES_CONFIG = [
83 (TEXTURE_FRONT, 625, 455, 1.0, 1.4, 200, 0, 1, 0.01),
84 (TEXTURE_FRONT_RIGHT, 705, 445, 1.0, 1.4, 200, 0, 0, 0.01),
85 (TEXTURE_3, 790, 410, 0.8, 1.4, 220, 1, 1, 0.01),
86 (TEXTURE_4, 855, 415, 1.5, 1.4, 200, 0, 0, 0.01),
87 (TEXTURE_5, 910, 385, 1.5, 1.4, 200, 0, 0, 0.01),
88 (TEXTURE_6, 940, 345, 1.0, 1.4, 200, 0, 0, 0.01),
89 (TEXTURE_7, 970, 275, 1.2, 1.4, 200, 0, 0, 0.01),
90 (TEXTURE_8, 960, 250, 1.2, 2.1, 200, 0, 0, 0.01),
91 (TEXTURE_9, 930, 220, 1.3, 2.1, 200, 0, 0, 0.01),
92 (TEXTURE_10, 885, 175, 1.0, 2.1, 200, 0, 0, 0.01),
93 (TEXTURE_11, 830, 160, 1.2, 2.1, 200, 0, 0, 0.01),
94 (TEXTURE_12, 760, 125, 1.2, 2.1, 200, 0, 0, 0.01),
95 (TEXTURE_13, 615, 120, 1.2, 2.1, 200, 0, 0, 0.01),
96 (TEXTURE_14, 535, 125, 1.2, 2.1, 200, 0, 0, 0.01),
97 (TEXTURE_15, 460, 142, 1.2, 2.1, 200, 0, 0, 0.01),
98 (TEXTURE_16, 400, 178, 1.2, 2.1, 200, 0, 0, 0.01),
99 (TEXTURE_17, 350, 200, 1.2, 2.1, 200, 0, 0, 0.01),
100 (TEXTURE_18, 315, 240, 1.3, 2.1, 200, 0, 0, 0.01),
101 (TEXTURE_185, 322, 290, 1.3, 2.1, 200, 0, 0, 0.01),
102 (TEXTURE_19, 305, 310, 1.3, 2.1, 200, 0, 0, 0.01),
103 (TEXTURE_20, 303, 345, 1.0, 2.1, 200, 0, 0, 0.01),
104 (TEXTURE_21, 350, 405, 1.1, 1.6, 200, 0, 0, 0.01),
105 (TEXTURE_22, 410, 425, 1.1, 2.1, 200, 0, 0, 0.01),
106 (TEXTURE_23, 480, 455, 1.1, 2.1, 180, 0, 0, 0.01),
107 (TEXTURE_24, 555, 460, 1.1, 2.1, 180, 0, 0, 0.01),

```

```

108 ]
109 # ключ текстуры, позиция(x,y), масштаб, статический поворот, амплитуда дрожания,
110 # непрозрачность, радиус размытия, итерации размытия, амплитуда случайного
    изменения масштаба
111
112
113 class FlameLayer:
114     def __init__(self, texture, base_x, base_y, scale, shake_amp,
115 alpha=255, blur_radius=0, blur_iterations=1,
116 scale_variation=0):
117 self.original_texture = texture
118 self.base_x = base_x
119 self.base_y = base_y
120 self.scale = scale
121 self.base_scale = scale
122 self.scale_variation = scale_variation
123 self.shake_amp = shake_amp
124 self.alpha = alpha
125 self.blur_radius = blur_radius
126 self.blur_iterations = blur_iterations
127 self.offset_x = 0
128 self.offset_y = 0
129
130 def update_turbulence(self):
131 self.offset_x = random.uniform(-self.shake_amp, self.shake_amp)
132 self.offset_y = random.uniform(-self.shake_amp, self.shake_amp)
133 if self.scale_variation > 0:
134 self.scale = random.uniform(self.base_scale - self.scale_variation,
135 self.base_scale + self.scale_variation)
136
137 def render(self):
138 w, h = self.original_texture.get_size()
139 new_w = max(1, int(w * self.scale))
140 new_h = max(1, int(h * self.scale))
141 scaled = pygame.transform.scale(self.original_texture, (new_w, new_h))
142 if self.blur_radius > 0:
143 scaled = apply_blur_approximation(scaled, self.blur_radius, self.
    blur_iterations)
144 if self.alpha < 255:
145 scaled.set_alpha(self.alpha)
146 return scaled
147
148 def get_screen_position(self):
149 return (self.base_x + self.offset_x, self.base_y + self.offset_y)
150
151
152 class SparkParticle:
153     def __init__(self, texture, x, y, scale, duration, max_alpha,
    blur_radius=0, blur_iterations=1):
154 self.original_texture = texture
155 self.x = x
156 self.y = y
157 self.scale = scale
158 self.duration = duration
159 self.max_alpha = max_alpha
160 self.blur_radius = blur_radius

```

```

161 self.blur_iterations = blur_iterations
162 self.age = 0.0
163
164 def update(self, dt):
165     self.age += dt
166     return self.age < self.duration
167
168 def get_alpha(self):
169     t = self.age / self.duration
170     if t <= 0.5:
171         alpha_factor = t * 2
172     else:
173         alpha_factor = 2 - t * 2
174     return int(self.max_alpha * alpha_factor)
175
176 def render(self):
177     w, h = self.original_texture.get_size()
178     new_w = max(1, int(w * self.scale))
179     new_h = max(1, int(h * self.scale))
180     scaled = pygame.transform.scale(self.original_texture, (new_w, new_h))
181     if self.blur_radius > 0:
182         scaled = apply_blur_approximation(scaled, self.blur_radius, self.
            blur_iterations)
183     scaled.set_alpha(self.get_alpha())
184     return scaled
185
186 def draw(self, screen):
187     surf = self.render()
188     rect = surf.get_rect(center=(self.x, self.y))
189     screen.blit(surf, rect)
190
191 def apply_blur_approximation(surface, radius, iterations):
192     if radius <= 0:
193         return surface
194     w, h = surface.get_size()
195     small_w = max(1, w // (radius * 2 + 1))
196     small_h = max(1, h // (radius * 2 + 1))
197     small = pygame.transform.smoothscale(surface, (small_w, small_h))
198     for _ in range(iterations):
199         small = pygame.transform.smoothscale(small, (small_w, small_h))
200     blurred = pygame.transform.smoothscale(small, (w, h))
201     return blurred
202
203 def load_sprite(path):
204     try:
205         img = pygame.image.load(path).convert_alpha()
206         return img
207     except Exception as e:
208         surf = pygame.Surface((100, 100), pygame.SRCALPHA)
209         pygame.draw.circle(surf, (0, 100, 200, 200), (50, 50), 50)
210         return surf
211
212 def initialize_flame_layers(configs, textures, size_coef):
213     instances = []
214     for (tex_key, x, y, scale, shake_amp, alpha,
215         blur_rad, blur_iter, scale_var) in configs:

```

```

216 tex = textures[tex_key]
217 instances.append(FlameLayer(
218 texture=tex,
219 base_x=x * size_coef,
220 base_y=y * size_coef,
221 scale=scale * size_coef,
222 shake_amp=shake_amp * size_coef,
223 alpha=alpha,
224 blur_radius=blur_rad,
225 blur_iterations=blur_iter,
226 scale_variation=scale_var * size_coef
227 ))
228 return instances
229
230 def main():
231 pygame.init()
232 pygame.display.set_mode((1, 1), pygame.NOFRAME)
233 try:
234 bg = pygame.image.load(BACKGROUND_IMAGE).convert()
235 bg = pygame.transform.scale(bg, (bg.get_width() * SIZE_COEF, bg.
    get_height() * SIZE_COEF))
236 except Exception as e:
237 print(f"Ошибка загрузки фона: {e}")
238 pygame.quit()
239 return
240 window_size = bg.get_size()
241 screen = pygame.display.set_mode(window_size)
242 texture_files = {
243     TEXTURE_FRONT: TEXTURE_FRONT,
244     TEXTURE_FRONT_RIGHT: TEXTURE_FRONT_RIGHT,
245     TEXTURE_3: TEXTURE_3,
246     TEXTURE_4: TEXTURE_4,
247     TEXTURE_5: TEXTURE_5,
248     TEXTURE_6: TEXTURE_6,
249     TEXTURE_7: TEXTURE_7,
250     TEXTURE_8: TEXTURE_8,
251     TEXTURE_9: TEXTURE_9,
252     TEXTURE_10: TEXTURE_10,
253     TEXTURE_11: TEXTURE_11,
254     TEXTURE_12: TEXTURE_12,
255     TEXTURE_13: TEXTURE_13,
256     TEXTURE_14: TEXTURE_14,
257     TEXTURE_15: TEXTURE_15,
258     TEXTURE_16: TEXTURE_16,
259     TEXTURE_17: TEXTURE_17,
260     TEXTURE_18: TEXTURE_18,
261     TEXTURE_185: TEXTURE_185,
262     TEXTURE_19: TEXTURE_19,
263     TEXTURE_20: TEXTURE_20,
264     TEXTURE_21: TEXTURE_21,
265     TEXTURE_22: TEXTURE_22,
266     TEXTURE_23: TEXTURE_23,
267     TEXTURE_24: TEXTURE_24,
268 }
269 textures = {path: load_sprite(path) for path in texture_files.values()}
270 flames = initialize_flame_layers(FLAMES_CONFIG, textures, SIZE_COEF)

```

```

271 spark_textures = {}
272 for cfg in SPARKS_CONFIG:
273     path = cfg[1]
274     if path not in spark_textures:
275         spark_textures[path] = load_sprite(path)
276     sparks = []
277     for cfg in SPARKS_CONFIG:
278         start_time, tex_path, x, y, scale, duration, max_alpha, blur_radius,
            blur_iter = cfg
279     tex = spark_textures[tex_path]
280     spark = SparkParticle(tex, x * SIZE_COEF, y * SIZE_COEF, scale, duration
        , max_alpha, blur_radius, blur_iter)
281     sparks.append((start_time, spark))
282     video_writer = None
283     record_enabled = RECORD_VIDEO
284     if record_enabled:
285         try:
286             import cv2
287             fourcc = cv2.VideoWriter_fourcc(*'mp4v')
288             video_writer = cv2.VideoWriter(VIDEO_OUTPUT, fourcc, VIDEO_FPS,
                window_size)
289             print(f"Запись видео в {VIDEO_OUTPUT} размер( {window_size})")
290         except ImportError:
291             record_enabled = False
292         except Exception as e:
293             print(f"Ошибка инициализации видеозаписи: {e}")
294             record_enabled = False
295     clock = pygame.time.Clock()
296     start_time = time.time()
297     running = True
298     active_sparks = []
299     frame_count = 0
300     while running:
301         dt = clock.tick(FPS) / 1000.0
302         current_time = time.time() - start_time
303         if DURATION > 0 and current_time > DURATION:
304             break
305         for event in pygame.event.get():
306             if event.type == pygame.QUIT:
307                 running = False
308         for flame in flames:
309             flame.update_turbulence()
310         for (s_time, spark) in sparks[:]:
311             if current_time >= s_time:
312                 active_sparks.append(spark)
313                 sparks.remove((s_time, spark))
314         for spark in active_sparks[:]:
315             if not spark.update(dt):
316                 active_sparks.remove(spark)
317         screen.blit(bg, (0, 0))
318         for flame in flames:
319             surf = flame.render()
320             x, y = flame.get_screen_position()
321             rect = surf.get_rect(center=(x, y))
322             screen.blit(surf, rect)
323         for spark in active_sparks:

```

```
324 spark.draw(screen)
325 pygame.display.flip()
326 if record_enabled and video_writer is not None:
327     frame = pygame.surfarray.array3d(screen)
328     frame = frame.swapaxes(0, 1)
329     frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
330     video_writer.write(frame)
331     frame_count += 1
332
333 if video_writer is not None:
334     video_writer.release()
335 pygame.quit()
336 if __name__ == "__main__":
337     main()
```